Experiential Learning
Project Assessment-Ion
"Music Playlist Management System"

Under the course

Python Programming Language (N-PCCCM304P)

Presented By
Samruddhi Varma

Under the Guidance
of Mrs. Mayuri Getme

Session: 2026–27 (ODD)

Department of CSE(AI&ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR

An Autonomous Institute, Affiliated to RTMNU, Nagpur

Contents

- Problem statement
- Objectives
- Algorithm
- Flowchart / pseudocode
- Understanding of Python logic
- GitHub account, repository creation, uploading code, meaningful commits, collaboration, individual contribution

Edit with WPS Office

1. Problem Statement

The Need

Managing a growing music collection with ordinary lists can make insertion and deletion less flexible. A linked-list based playlist can dynamically connect songs and update the playlist without shifting other elements.

Chosen Structure

Doubly Linked List
Each song stores:

- title
- artist
- previous link
- next link

Add

Insert a new song at the required position or at the end.

Delete / Search

Remove an unwanted song and locate a song by title or artist.

Traverse

Move forward or backward through the playlist efficiently.

2. Objectives

Dynamic playlist

Store songs as linked nodes so the playlist can grow or shrink at runtime.

Add songs

Insert songs at the beginning, end, or a selected position.

Delete songs

Remove a song by title or position and reconnect neighboring nodes.

Search songs

Find a song quickly by comparing the required key with node data.

Traverse playlist

Display songs from first to last and, with a doubly list, from last to first.

Apply Python logic

Use classes, objects, references, loops and conditional statements to implement the system.

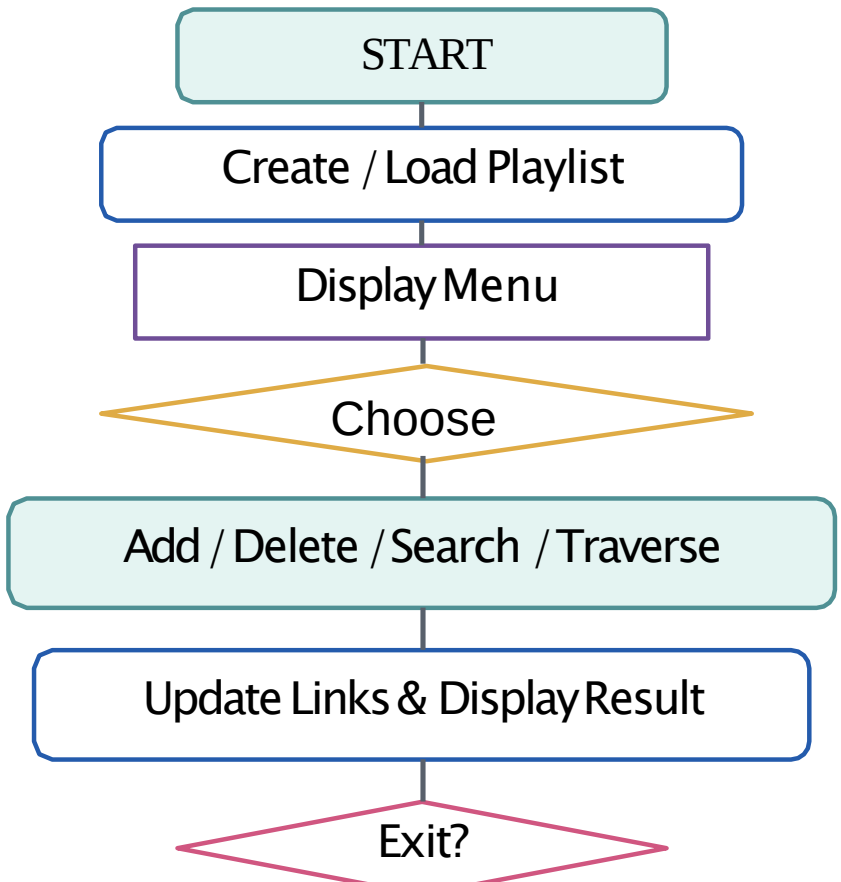
3. Algorithm

- 1. Create a Node class containing song data, prev and next references.
- 2. Create a Playlist class with head and tail pointers.
- 3. ADD: create a node, connect it with adjacent nodes, then update head/tail.
- 4. DELETE: search for the target node, reconnect its previous and next nodes, then remove the link.
- 5. SEARCH: start at head and compare each node's title/artist until a match is found.
- 6. TRAVERSE: visit nodes from head → tail; reverse traversal uses tail → head.
- 7. Repeat menu operations until the user selects Exit.

4. Flowchart / Pseudocode

Pseudocode

START
↓
Initialize playlist
↓
Read choice
↓
IF Add → insert node
IF Delete → remove node
IF Search → compare data
IF Traverse → visit nodes
↓
IF Exit → STOP
ELSE → repeat menu



5. Understanding of Python Logic

```
class Node:
    data =song
    prev =None
    next =None
```

A node represents one song and stores links to neighbour

Playlist Class

head → first song
tail → last song

Methods can handle add(), delete(), search() and traverse().

```
while current is not None:
    check current.data
    current =current.next
```

Used for searching and traversal.

Conditions

```
if current.data ==target:
    song found
```

Conditions decide where to insert, delete or stop.

Why Linked List?

Nodes are connected by references, so insertion/deletion mainly requires changing links instead of shifting a whole array.

6. GitHub Account, Repository & Collaboration

1. GitHub Account

Create/sign in to GitHub and keep the project under a clear profile.

2. Repository

Create a repository such as:
Music-Playlist-Management
-System
Add README and Python source file.

3. Upload Code

Upload .py files and project documentation. Keep filenames meaningful.

4. Meaningful Commits

Examples:
"Added Node class"
"Implemented delete operation"
"Added reverse traversal"

5. Collaboration

Use branches or shared contributions, review changes, and merge tested work into the main branch.

6. Individual Contribution

Each member should clearly document the part they implemented, tested, or improved.

Thank You